

0.1 Pass by Reference vs Pass by Value

- Pass by Value: When a function is called, the actual parameter's value is copied into the function's formal parameter. Changes made to the formal parameter do not affect the actual parameter.
- Pass by Reference: Instead of passing a copy of the actual parameter, a reference (or address) to the actual parameter is passed. Changes made to the formal parameter will affect the actual parameter.

```
#include <iostream>
using namespace std;
void changeValue(int& a, int b) {
    a = 20; // This will change the original variable
    b = 30; // This will not change the original variable
}

int main() {
    int x = 10;
    int y = 10;
    changeValue(x, y);
    cout << "x: " << x << endl; // Outputs: x: 20
    cout << "y: " << y << endl; // Outputs: y: 10
    return 0;
}
```

0.2 Built-in Functions

Function Name	Purpose/What It Does
<code>sqrt(x)</code>	square root of x.
<code>abs(x)</code>	absolute value of x.
<code>sin(x)</code>	sine of angle x radians.
<code>cos(x)</code>	cosine of angle x radians.
<code>tan(x)</code>	tangent of angle x radians.
<code>log(x)</code>	natural logarithm (base e) of x.
<code>log2(x)</code>	logarithmic value with base 2 of x.
<code>log10(x)</code>	logarithmic value with base 10 of x.
<code>getline(cin, S)</code>	Reads a line of text from the input stream (<code>cin</code>) and stores it in the variable S.
<code>setprecision(x)</code>	Used to control the number of significant digits in floating-point or double values. If used with the <code>fixed</code> manipulator, it controls the number of digits after the decimal point .
<code>max(x, y, ...)</code>	maximum of all the parameters given. (All parameters must have the same data type).
<code>min(x, y, ...)</code>	minimum of all the parameters given. (All parameters must have the same data type).

Built-in Functions

Function Name	Purpose/What It Does
<code>sort(arr, arr+len)</code> OR <code>sort(vec.begin(), vec.end())</code>	Sorts arrays or containers in ascending order. It can take an optional third parameter to sort in descending order <code>greater<>()</code> .
<code>reverse(arr, arr+len)</code>	Reverses the order of elements in a container.
<code>next_permutation(arr, arr+len)</code>	Gets the next lexicographically smallest permutation of the container it is used on. This function is most commonly used in a <code>do-while</code> loop to try all possible orders of an array, provided the array is sorted beforehand.
<code>count(arr, arr+len, x)</code>	Used to count the number of occurrences of a particular element <code>x</code> within a range.

```
#include <bits/stdc++.h>
using namespace std;

#define fast_io \
    ios::sync_with_stdio(false); \
    cin.tie(nullptr);
#define endl '\n'

int main()
{
    fast_io;

    int arr[5] = {2, 1, 33, 2, 45};
    // using `greater<>()` to sort in descending order
    sort(arr, arr + 5, greater<>());
    cout << "arr:" << endl;
    for (auto &i : arr)
        cout << i << endl;

    int arr2[5] = {2, 1, 33, 2, 45};
    // using lambda expression to sort in descending order
    sort(arr2, arr2 + 5, [](int a, int b)
        { return a > b; });
    cout << "arr2:" << endl;
    for (auto &i : arr2)
        cout << i << endl;

    cout << "max = " << max({2, 1, 33, 2, 45}) << endl;
    cout << "*max_element = " << *max_element(arr, arr + 5) << endl;

    int arr3[3] = {1, 2, 3};
    cout << "arr3 permutations:" << endl;
    for (auto &i : arr3)
        cout << i << " "; // 1 2 3
```

```
next_permutation(arr3, arr3 + 3);
for (auto &&i : arr3)
    cout << i << " "; // 1 3 2

next_permutation(arr3, arr3 + 3);
for (auto &&i : arr3)
    cout << i << " "; // 2 1 3

next_permutation(arr3, arr3 + 3);
for (auto &&i : arr3)
    cout << i << " "; // 2 3 1

return 0;
}
```